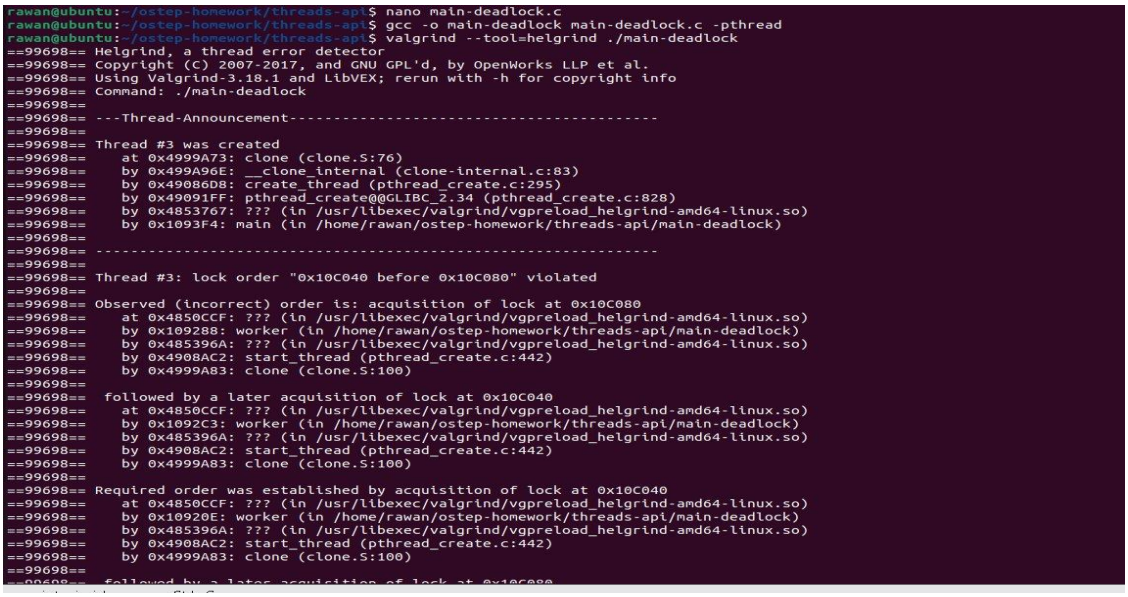
Lab 10: Threads and Locks

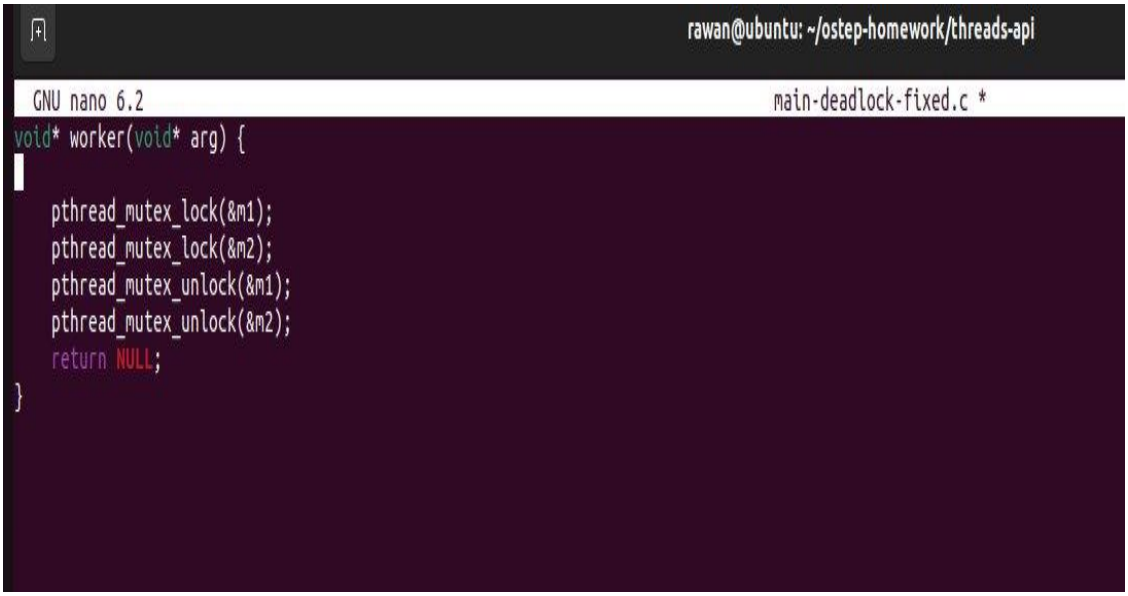
This report explains common threading problems in Operating Systems using simple examples. The experiments were tested using Helgrind to detect synchronization issues.

Part 1: Race Condition Example

The first program contains a shared variable called *balance*. Both the main thread and worker thread update this variable at the same time without protection. Because of this, the final result may become incorrect.

Helgrind was executed to check the program behavior. The tool detected a race condition and showed that two threads accessed the same memory location without synchronization.



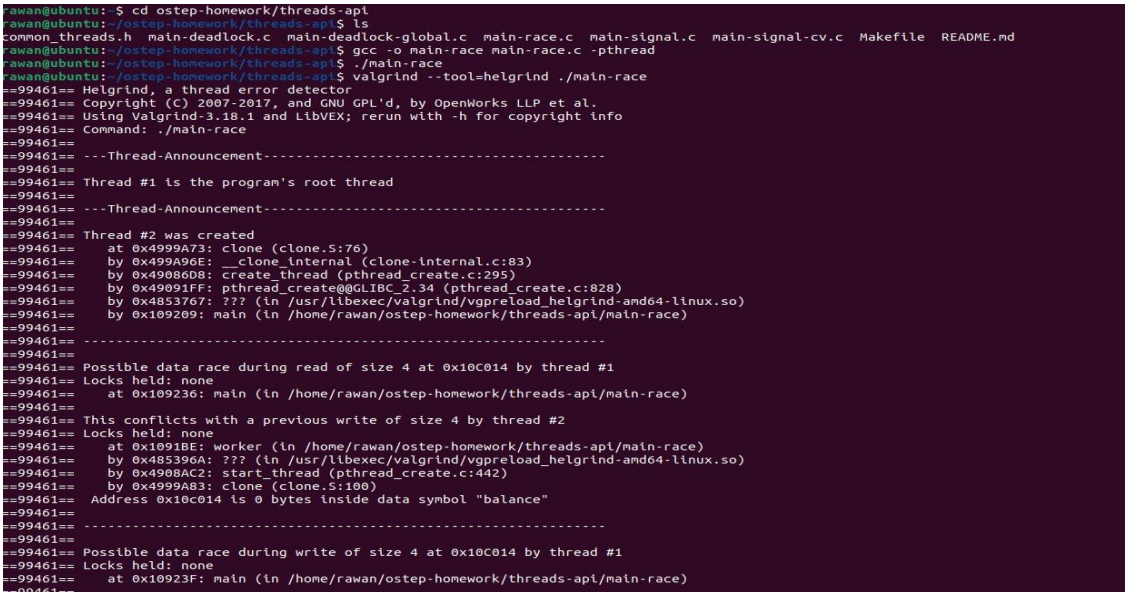


Fixing the Problem
A mutex lock was added around the shared variable access. This allows only one thread to update the variable at a time and prevents incorrect results.

Part 2: Deadlock Example

The second program uses two mutexes and two threads. Each thread locks the mutexes in a different order. This situation may cause both threads to wait forever, which is known as a deadlock.

Helgrind reported a lock ordering issue and warned about a possible deadlock between the two threads.



Deadlock Solution
The issue was solved by forcing both threads to lock the mutexes in the same order every time. This removes the circular waiting problem.

Conclusion
This lab demonstrated how synchronization errors can happen in multithreaded programs. Using mutexes and proper lock ordering helps avoid race conditions and deadlocks.